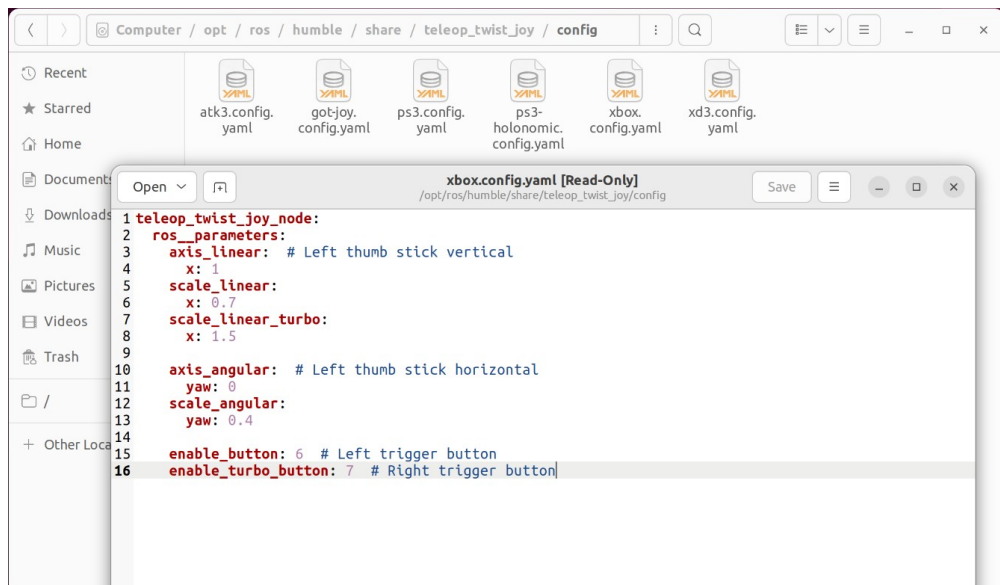


Assignment 5 Joystick Controls TurtleSim

This tutorial describes how to use a joystick to control the turtle's movement in the turtlesim node.

Prerequisite packages:

joy, teleop_twist_joy and turtlesim



1. We have to create joystick parameter configuration file for customising your button control.

Go to folder “opt/ros/humble/share/teleop_twist_joy/config”, then open the config.yaml file that same as your joystick type. For this tutorial, I used Xbox joystick, so I opened and copied all parameters from the xbox.config.yaml file same as above image.

```

rorgot@rorgot-laptop: ~/HM4
rorgot@rorgot-laptop:~$ mkdir HM4 && cd HM4
rorgot@rorgot-laptop:~/HM4$
  
```

2. Create a new folder for this tutorial and change directory to its.

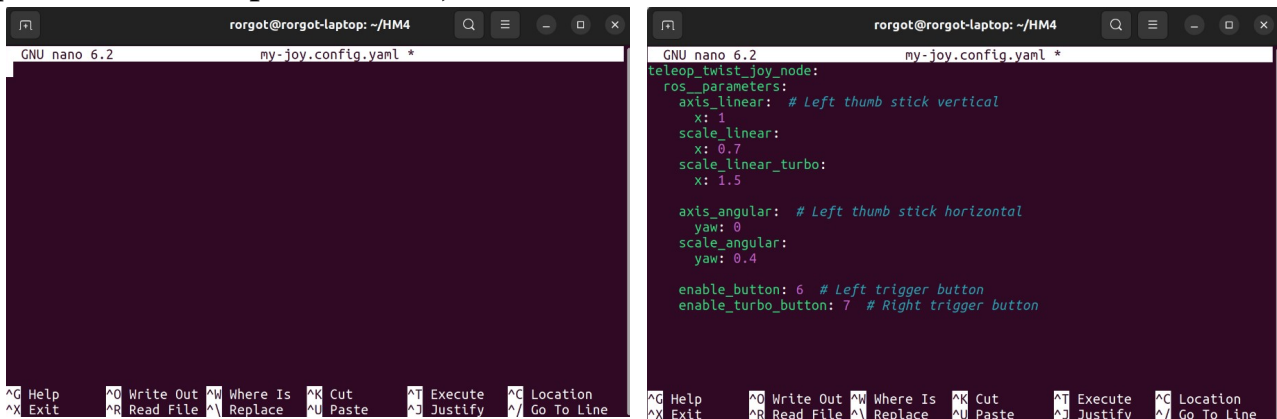
mkdir HM4 && cd HM4

```
rorgot@rorgot-laptop: ~/HM4$ sudo nano my-joy.config.yaml
[sudo] password for rorgot:
```

3. Create file my-joy.config.yaml in HM4 folder.

```
sudo nano my-joy.config.yaml
```

(After use sudo command you may have to enter your ubuntu password then press enter.)



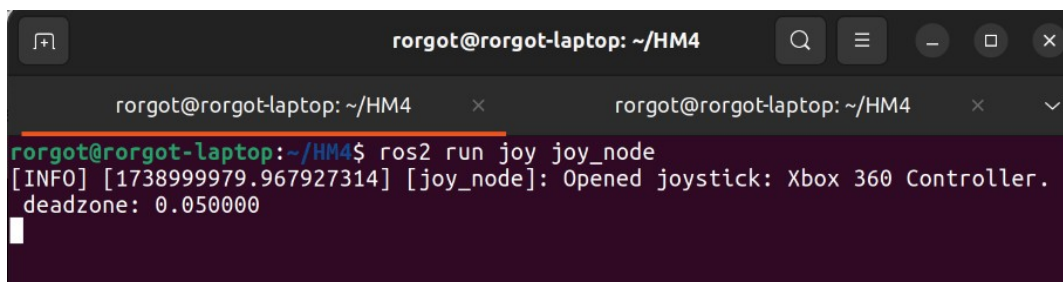
4. The my-joy.config.yaml file will appear in your terminal (Left image). You have to paste parameters from the xbox.config.yaml file, which copied before (Right image is result after paste).

The parameters meaning from the right image

- **axis_linear:** linear movement control by axes (analog) button. It can split into 3 parameters: axis_linear.x, axis_linear.y, axis_linear.z. For control the turtlesim use only axis_linear.x.

- **axis_angular:** angular movement control by axes (analog) button. It can split into 3 parameters: axis_angular.yaw, axis_angular.pitch, axis_angular.roll. For control the turtlesim use only axis_angular.yaw.

- **scale_linear and scale_angular:** define normal linear and angular velocity of turtle respectively. They also can split into its main axis (linear: x, y, z / angular: yaw, pitch, roll)
- **scale_linear_turbo:** define the linear turbo velocity of the turtle. They also can split into its main axis (linear: x, y, z / angular: yaw, pitch, roll)
- **enable_button:** the button that you have to press it along with the movement to enable the movement.
- **enable_turbo_button:** the button that you have to hold it when you need to boost the turtle velocity.



```
rorgot@rorgot-laptop: ~/HM4
rorgot@rorgot-laptop: ~/HM4$ ros2 run joy joy_node
[INFO] [1738999979.967927314] [joy_node]: Opened joystick: Xbox 360 Controller.
deadzone: 0.050000
```

5. Check your joystick buttons and axes address number to define the movement control, normal speed, and enable buttons.

Open a new terminal, Run joy node.

ros2 run joy joy_node



```
frame_id: joy
axes:
  Address
  -0.0 0
  -0.0 1
  1.0 2
  -0.0 3
  -0.0 4
  1.0 5
  0.0 6
  0.0 7
  0.0 8
buttons:
  -0 0
  -0 1
  -0 2
  -0 3
  -0 4
  -0 5
  -0 6
  -0 7
  -0 8
  -0 9
  -0 10
---
```

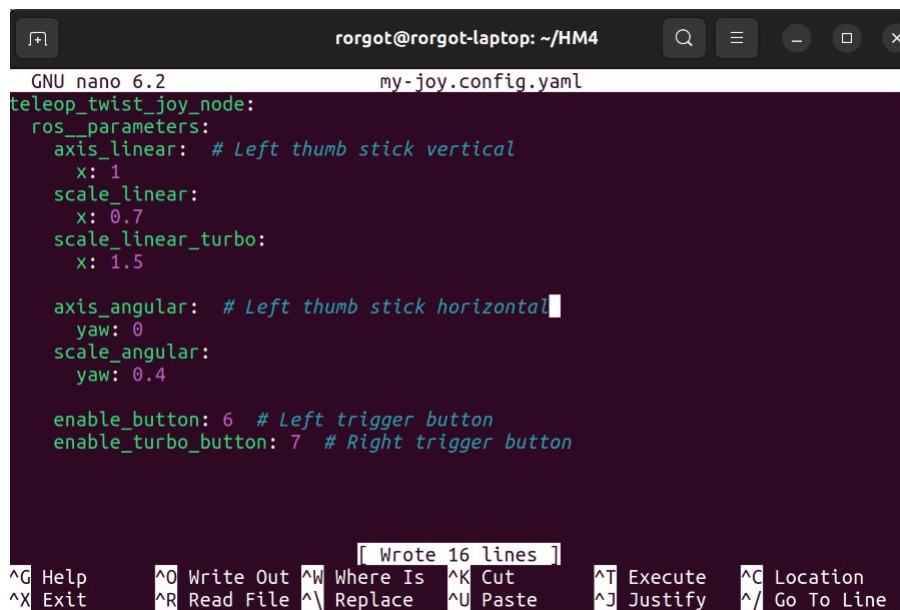
Axes for control the movement.

Button for enable/trigger

Open a new terminal, Run

ros2 topic echo joy

To see the buttons and axes address number of your joystick by press or move the button and see the result from the address value same as above image. Then you have to select the button you need before next step.



```

rorgot@rorgot-laptop: ~/HM4
GNU nano 6.2 my-joy.config.yaml
teleop_twist_joy_node:
  ros__parameters:
    axis_linear: # Left thumb stick vertical
      x: 1
    scale_linear:
      x: 0.7
    scale_linear_turbo:
      x: 1.5

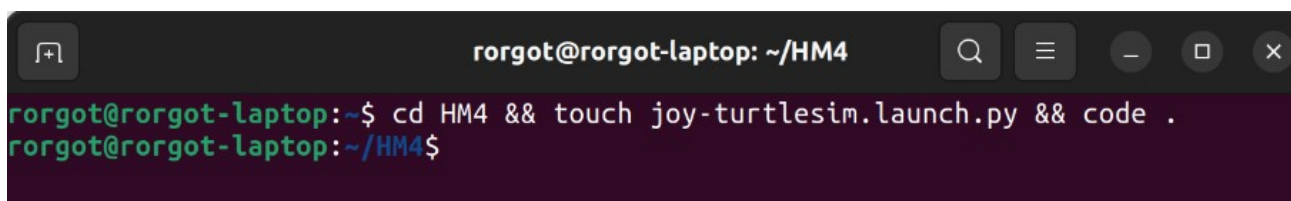
    axis_angular: # Left thumb stick horizontal
      yaw: 0
    scale_angular:
      yaw: 0.4

    enable_button: 6 # Left trigger button
    enable_turbo_button: 7 # Right trigger button
  
```

Wrote 16 lines

Help Write Out Where Is Cut Execute Location
Exit Read File Replace Paste Justify Go To Line

6. Go back to my-joy.config.yaml terminal. Change the parameters value. For this tutorial, I will use the value same as above image. Then press ctrl+s to save and ctrl+x to close this file. And close all terminals.



```

rorgot@rorgot-laptop: ~/HM4
rorgot@rorgot-laptop:~$ cd HM4 && touch joy-turtlesim.launch.py && code .
rorgot@rorgot-laptop:~/HM4$
  
```

7. Open new terminal, change directory to HM4, then create joy-turtlesim.launch.py file and use vscode to edit the script in its.

cd HM4 && touch joy-turtlesim.launch.py && code .

```

joy-turtlesim.launch.py - HM4 - Visual Studio Code
File Edit Selection View Go Run Terminal Help

EXPLORER
HM4
  > __pycache__
  joy-turtlesim.launch.py
  ! my-joy.config.yaml

joy-turtlesim.launch.py X
joy-turtlesim.launch.py > generate_launch_description
1 from launch import LaunchDescription
2 from launch_ros.actions import Node
3 from launch.actions import ExecuteProcess
4
5 def generate_launch_description():
6     return LaunchDescription([
7         Node(                                     #Create turtle1 node
8             package='turtlesim',
9             executable='turtlesim_node',
10            name='turtle1'
11        ),
12        Node(                                     #Create turtle2 node
13            package='turtlesim',
14            executable='turtlesim_node',
15            name='turtle2'
16        ),
17        Node(                                     #Create joy node
18            package='joy',
19            executable='joy_node',
20        ),
21        Node(                                     #Create teleop_twist_joy_node map joy value send to turtlesim
22            package='teleop_twist_joy',
23            executable='teleop_node',
24            parameters=['my-joy.config.yaml'], #It will use my-joy.config.yaml in this folder
25            remappings=[
26                ('/cmd_vel', 'turtle1/cmd_vel'), #Make /cmd_vel topic publish to turtle1/cmd_vel topic instead,
27                                                  #because the turtle1 and turtle2 node subscribed to the turtle1/cmd_vel not /cmd_vel
28            ]
29        )
30    ])
31

```

8. Copy this code and paste to your joy-turtlesim.launch.py file.

```

from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import ExecuteProcess

def generate_launch_description():
    return LaunchDescription([
        Node(                                     #Create turtle1 node
            package='turtlesim',
            executable='turtlesim_node',
            name='turtle1'
        ),
        Node(                                     #Create turtle2 node
            package='turtlesim',
            executable='turtlesim_node',
            name='turtle2'
        ),
        Node(                                     #Create joy node
            package='joy',
            executable='joy_node',
        ),
        Node(                                     #Create teleop_twist_joy_node map joy value send to turtlesim
            package='teleop_twist_joy',
            executable='teleop_node',
            parameters=['my-joy.config.yaml'], #It will use my-joy.config.yaml in this folder
            remappings=[
                ('/cmd_vel', 'turtle1/cmd_vel'), #Make /cmd_vel topic publish to turtle1/cmd_vel topic instead,
                                                  #because the turtle1 and turtle2 node subscribed to the turtle1/cmd_vel not /cmd_vel
            ]
        )
    ])

```

```

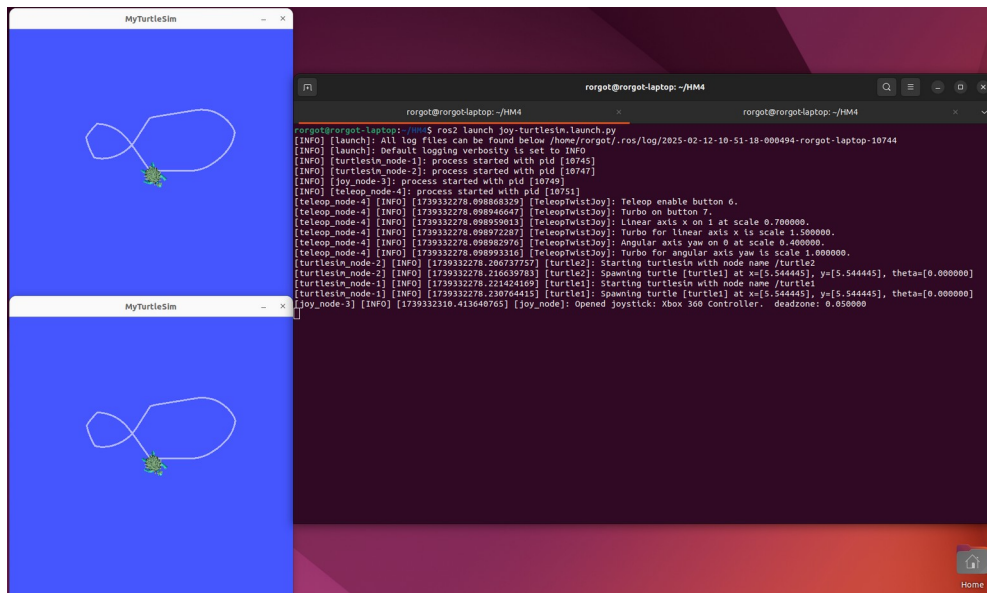
rorgot@rorgot-laptop:~/HM4$ ros2 node info /turtle1
/turtle1
Subscribers:
 /parameter_events: rcl_interfaces/msg/ParameterEvent
 /turtle1/cmd_vel: geometry_msgs/msg/Twist
Publishers:
 /parameter_events: rcl_interfaces/msg/ParameterEvent
 /rosout: rcl_interfaces/msg/Log
 /turtle1/color_sensor: turtlesim/msg/Color
 /turtle1/pose: turtlesim/msg/Pose
Service Servers:
 /clear: std_srvs/srv/Empty
 /kill: turtlesim/srv/Kill
 /reset: std_srvs/srv/Empty
 /spawn: turtlesim/srv/Spawn
 /turtle1/describe_parameters: rcl_interfaces/srv/DescribeParameters
 /turtle1/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
 /turtle1/get_parameters: rcl_interfaces/srv/GetParameters
 /turtle1/list_parameters: rcl_interfaces/srv/ListParameters
 /turtle1/set_parameters: rcl_interfaces/srv/SetParameters
 /turtle1/set_parameters_atomically: rcl_interfaces/srv/SetParametersAtomically
Action Servers:
 /turtle1/rotate_absolute: turtlesim/action/RotateAbsolute
Action Clients:

```

*We use remappings in teleop_twist_joy node to redirect /cmd_vel to turtle1/cmd_vel because the turtle1 node subscribes to turtle1/cmd_vel, not /cmd_vel. (We can confirm this by run the turtlesim node's, then checking subscribed topics by `ros2 node info <node_name>`, as shown in the image above.)

9. Save your joy-turtlesim.launch.py file and go back to terminal HM4. Run

`ros2 launch joy-turtlesim.launch.py`



After run the command, 2 turtlesim windows will appear and we can use joystick to control the turtle movement. If we see the info log after run the command, your joystick button setting will appear at [teleop_node] line, which may confirm your setting from it.